

Saab - Summer project: Spectrum analyser

Andreas Söderlund
Oliver Eriksson

August 15, 2025

Acronyms

AD9361	Transceiver on the SDR
ADC	Analogue-to-digital converter
ANTSDR e200	Model of SDR
API	Application programming interface
AXI	Advanced eXtensible Interface
AXIS	AXI-Stream
CPU	Central Processing Unit
DAC	Digital-to-analogue converter
DMA	Direct Memory Access
DSP	Digital signal processing
FFT	Fast Fourier Transform
FIFO	First in First out
FPGA	Field Programmable Gate Array
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HDL	Hardware description language
HP port	High-performance port
IC	Integrated Circuit
IIO	Industrial input output
IP core	Intellectual Property core
JSON	JavaScript Object Notation
PL	Programmable Logic
PS	Processing System
RF	Radio Frequency
RFIC	Radio Frequency Integrated Circuit
SDR	Software Defined Radio
SoC	System on Chip

Introduction

A Spectrum analyser is an electronic instrument that measures the power of the spectrum of an input signal versus frequency within the frequency range of the instrument. The spectrum analyser is the frequency counterpart to the oscilloscope, which captures and displays the behaviour of an electrical signal over time.

The basis of analysing signals in the frequency domain is the Fourier transform. This conversion is made digitally by the Fast Fourier Transform (FFT) algorithm, which is an algorithm that can be implemented in a system's different levels of execution. Implementations in software run on a Central Processing Unit (CPU) or on a Graphics Processing Unit (GPU). Another way is to implement the algorithm in hardware using an Integrated Circuit (IC) or by running it on a Field Programmable Gate Array (FPGA) by writing Hardware

description language (HDL). The main difference between the execution levels is the speed in which the algorithm operates. The slowest alternative is generally the CPU followed by a performance improvement on the GPU, but the best result is a hardware implementation that is explicitly made for this task. This type of hardware is called an *accelerator*.

The project is a PoC, with the goal of building a spectrum analyser on the ANTSDR e200 Software Defined Radio (SDR). The purpose was to explore the possibility of implementing parts of DSP, most notably a FFT on the Programmable Logic (PL) part of the ZYNQ processor (that is the FPGA). The SDR should be able to receive an analogue signal, convert it to a digital signal and process it using a FFT on the hardware level. The SDR should then present the captured signal in a Graphical User Interface (GUI). The GUI in turn should be able to plot the received data and be able to send parameters to the SDR in order to control, for example, the carrier frequency to monitor.

Method

The project is comprised of two sections that have been developed separately and brought together to a working system. The first section, the hardware section, alters the FPGA hardware design to implement a FFT. Initially, the firmware used is intended for PlutoSDR. This is the base of the hardware section, and it allowed us to make changes to already working firmware.

The first goal with this firmware was to create a dummy filter in the hardware design. Upon successfully achieving this, the firmware was upgraded and loaded onto the SDR. The next goal was the FFT. Figuring out a clever way to implement this, and actually implementing it, is where most time has been spent. While pondering and testing hardware implementations, the other section, the software section, was being worked on. The samples are then sent to the GUI where they are displayed.

We had to establish a method to interchange data between the sdr and a host which runs the gui. We settled for creating a server that handles all communication between the SDR and GUI.

Specific implementation details such as interfaces, communication, and more are brought up in the subsequent Hardware design (-) and Software (??) sections.

Theory

Here, we go over some background theory of signals sampling, and a specific streaming protocol that were all used to make this project work.

SDR and sampling

The SDR has an AD9361 transceiver. Specifically, its receiver was used in this project, and most notably, the use of the radio spectrum.

In essence, the receiver is responsible for converting an incoming radio wave into an electric voltage which is stored digitally in 12-bit registers for further processing. This voltage is then sampled by an Analogue-to-digital converter (ADC), producing a digital representation of the radio wave.

In order to save a radio signal digitally or process it, it must be sampled at a finite rate. There are certain quantities to be concerned with, such as

- Sampling frequency f_S [MHz] - the number of times a system (the ADC) samples a signal per second.
- Sampling period $\tau_S = \frac{1}{f_S}$ [μs] - the regular interval in microseconds between each sample
- Bandwidth f_B - the maximum frequency component in a range of frequencies present in a signal.

The Nyquist-Shannon sampling theorem [1] states that a signal can be accurately reconstructed from its samples if the sampling frequency f_S is at least twice the max frequency component f_B of the original signal. This minimum sampling frequency is known as the Nyquist rate. The condition is

$$f_S > 2f_B \quad (1)$$

If the sampling frequency f_S is too low, aliasing occurs, where high-frequency components are misrepresented as lower frequencies, causing ambiguity in what the original signal is. If we know the signal can be expressed as a certain continuous function of time $f(t)$, then the theory of Fourier Series says that any such function can be expressed in terms of a sum of cosines and sines of different frequencies. Cos and sin are related by having a $\pi/2$ radians phase difference. At any moment in time, the signal has some amplitude A , and phase ϕ .

IQ sampling

Transmitting an RF signal $x = x(t)$ through some medium at a certain carrier frequency f_C involves controlling the signal's amplitude $A = A(t)$ and its phase $\phi = \phi(t)$. We can model the signal as

$$x = A \cos(2\pi f_C t - \phi) \quad (2)$$

In Radio Frequency (RF) circuitry, it is especially difficult to influence ϕ directly, so IQ sampling becomes necessary. IQ is a mathematical construction used to represent amplitude and phase of a signal with no loss of information. It uses the fact that the way $x(t)$ is written is not unique, it can be represented in equivalent forms. To see that, trigonometry gives

$$x = A \cos(2\pi f_C t - \phi) = A(\cos 2\pi f_C t \cos \phi + \sin 2\pi f_C t \sin \phi). \quad (3)$$

Now, we define

$$\begin{aligned} I &= A \cos \phi && \text{In phase} \\ Q &= A \sin \phi && \text{Quadrature} \end{aligned} \quad (4)$$

Since A and ϕ are functions of time, so are I and Q . Thinking in terms of algebra, the polar coordinate basis $(A \cos \phi, A \sin \phi)$ spans all of $\mathbb{R}^2$ space, which means any vector in the space can be represented as a linear combination thereof. Thus, the same applies for I and Q .

Now, equation (3) may be written as a linear combination of I and Q :

$$x = I \cos(2\pi f_C t) + Q \sin(2\pi f_C t) \quad (5)$$

This signal is transmitted on the carrier frequency f_C , but multiple signals that use this same f_C will have the same form as in equation (5), however, with unique I and Q .

Down-conversion

A signal described as equation (5) cannot be directly received by the Transceiver on the SDR (AD9361) transceiver. The SDR has a mixer that down-converts the signal in the frequency domain, from RF to center it around 0 Hz, that is, baseband, before being sampled by the ADC. Down-conversion is necessary, otherwise the signal would have to be sampled at a rate twice the carrier frequency, $2f_C$, which generally is out of reach for the ADC in the AD9361.

Following this down-conversion, the received signal is simply I and Q . When centered around 0 Hz, the maximum frequency is no longer f_C , as the carrier is removed, the signal's characteristics A and ϕ remain. Most signals we observed are around 100.0 kHz to 40.00 MHz wide in bandwidth, so through down-conversion, sampling can be done at a lower rate. Specifically, the AD9361 has a Radio Frequency Integrated Circuit (RFIC) that can sample signals with a bandwidth of up to 56.00 MHz, at a sample rate of up to 61.44 MHz.

In Figure 1, the LO is tuned to the signal's carrier frequency. The LO is then mixed with the incoming signal, and the two mixers of the RFIC shift the signal to $f_C - f_{LO}$ (baseband) (figure 2), causing down-conversion, producing I and Q , and finally sampling them in the ADC.

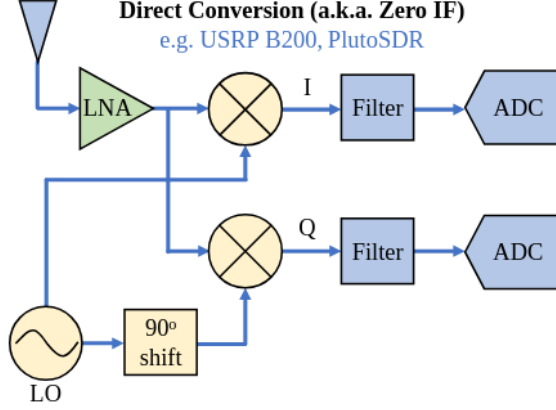


Figure 1: Diagram indicating how down-conversion works on the receiver.

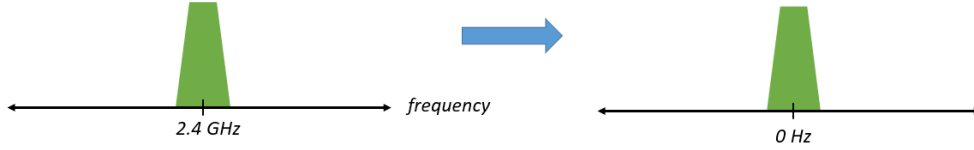


Figure 2: Down-conversion in the frequency domain.

Restoring the signal

To be able to reconstruct the sampled signal, the I and Q parts of (4) are treated as a complex number.

$$z = I + jQ \quad (6)$$

Now, finding the phase ϕ is done by computing the argument of z , and the amplitude A is the absolute value of z , i.e.

$$\begin{aligned} \arg z &= \arctan \frac{Q}{I} = \phi \\ |z| &= \sqrt{I^2 + Q^2} = \sqrt{A^2(\cos^2 \phi + \sin^2 \phi)} = A \end{aligned} \quad (7)$$

(7) are used for restoring the signal for plotting. As a bonus, the complex form allows for representing the signal in exponential form:

$$x = \Re\{(I + jQ)e^{j2\pi ft}\} = \Re\{Ae^{j(2\pi ft + \phi)}\} \quad (8)$$

The takeaway is that instead of sampling A and ϕ directly, the ADC samples the intermediates I and Q . This is because ϕ and A are difficult to sample directly, and IQ can still be used for modulation in hardware. When changing I and Q , we change the phase and amplitude of the signal at the carrier (modulating the carrier). Another option is to change the frequency of the carrier, that is, shift it slightly up or down, which is what FM radio does.

Frequency Domain

Equation (8) describes the signal without down-conversion. Such a signal is real. However, equation (6) shows the signal at baseband, and it is complex. This allows for representing negative frequencies. Plotting the frequency domain, the negative frequencies are the portion of the signal below the carrier frequency. This trait is only true for complex signals, a real signal would exhibit the same power at, let us say f and $-f$.

If the signal does not have any imaginary component, then Q is zero. This, in turn, means that the signal is only a sum of cosines without any phase shift. Such a function is symmetrical around the y-axis when plotting the frequency domain due to having precisely the same positive and negative components.

Fourier Transform

The FPGA logic computes a FFT using I and Q . The FFT is an algorithm that is based on the Discrete Fourier Transform (DFT), but it refactors terms in a creative way that makes use of recursion and improves computational complexity [2].

The time discrete Fourier Transform looks like

$$f_k = k/N,$$

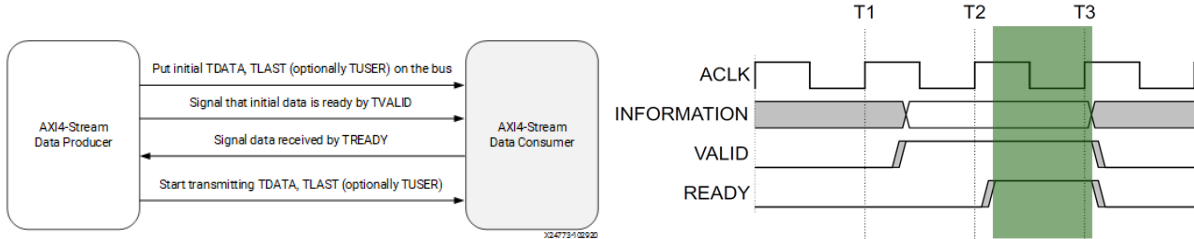
$$\mathcal{F}\{x(n)\}(f_k) = \sum_{n=0}^{N-1} x(n)e^{-i2\pi f_k n} \quad (9)$$

where N is the fft size.

AXI-Stream protocol

The AXI-Stream (AXIS) is part of the Advanced eXtensible Interface (AXI) protocol [3]. The AXI protocol is used for communication on ARM processors, System on Chip (SoC), and FPGA. The protocol outlines how data transfers should behave with handshakes to make sure both the sender and the receiver is ready for data transfer. The AXI4 protocol can send much data in bursts. When a transfer is complete, a "tlast" flag is sent along the last piece of data to indicate that a burst is completed.

AMD describes[4] the AXIS protocol as a streamlined, unidirectional protocol for high-speed transmissions. The protocol works on a ready and valid handshake. This handshake will make sure that there is valid information on the bus and that the receiver is ready for the transfer before the information is being sent. A transmission on the AXIS protocol will look like this: 1. Data is put on the bus, 2. The valid flag is raised to indicate that a sending operation is ready, 3. The receiver raises its ready flag when it can receive data, 4. Data is transferred when both valid and ready flags are raised. In Figure 3b depicts how the transfer of the data will occur on the positive edge of the clock marked with T3.



(a) Communication between two AXI-Stream devices. A handshake is made before data is transferred. [4] (b) AXI handshake: information is sent when both valid and ready are high. [3]

Figure 3: Overview of AXI-Stream handshake and transmission.

Hardware design

This section describes how the FPGA logic of the Zynq-7000 processor has been altered and what considerations were made in that process.

Firmware

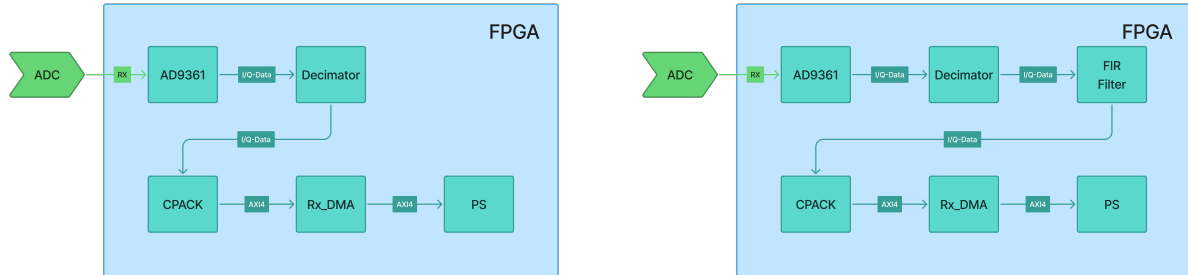
As previously described, signal processing is done on the digitised signal on ANTSDR e200, rather than processing by sending this to software on an external processor. This results in a small but useful analysis tool that could be extended for further purposes. The firmware we have used comes from the Microphase git repository "antsdr-fw-patch" [5]. This repository is in essence a collection of git sub-modules, containing build tools, operating system, and hardware design. When built together, they produce the PlutoSDR firmware, patched for the ANTSDR e200. The HDL sub-module contains several projects including relevant hardware design files. There are also drivers for the Linux kernel by Analog Devices, which is booted on the ZynqCPU.

Original design

The FPGA block design can be viewed in Vivado. In the block design, the most relevant signal path from the transceiver to the memory can be analysed. Figure 4a illustrates the block view of the RFIC and the IQ signal path from the ADC to the HP port of the Processing System (PS) therein.

More detailed, the AD9361 Intellectual Property core (IP core) receives the RX-signal as 12 bits from the RFIC that contains the ADC. The AD9361 widens the signals to 16-bit wide buses, which send I and Q respectively, with an accompanying valid signal. This valid signal is necessary since the signals on the FPGA are sent using a 100.0 MHz clock whilst the maximum sampling frequency is 61.44 MHz, meaning that the signal will not be valid on the I/Q buses for every clock cycle.

From the AD9361, the samples go through a Decimator IP core that can be controlled to either pass the signals through unchanged or through a filter that skips every 8th sample. From the Decimator, the I/Q sample goes to a CPACK[6] IP core. This block combines the I/Q samples to a 32-bit block to be sent to the Direct Memory Access (DMA) IP core. The latter[7] is the intermediate agent communicating with the processing system through its High-performance port (HP port). This port enables the DMA to write directly to the so called *DDR3* memory of the SDR.



(a) The signal path from the RFIC to be written to the HP port of the Zynq-7000 Processing system. (b) The signal path with applied FIR filter for testing purpose.

Figure 4: Signal path for original design and for first test design.

Updated Design

Thus far, we have provided a rundown on the workings of the original design. Our additions are incorporated in the design. As a first test, a FIR filter[8] was implemented, equipped with a dummy function, cancelling out all signals. The filter was applied to the signal that activates the Decimator. This first test design worked and can be seen in Figure 4b. The purpose of this test was to verify that the FPGA design could be manipulated without reverting upon build.

Upon establishing that the firmware could be changed. We added an FFT IP core[9] that is included in a Vivado library. The full signal path can be seen in Figure 5. To handle the continued I/Q data stream, and direct them as inputs to the FFT IP block, an intermediate *FFT buffer* was created that stored samples

and then sent them to the FFT using the AXIS protocol. The processed data from the FFT was received by a custom *FFT bin counter* IP core. This block took the FFT samples as input and passed along with each sample a corresponding bin number, analogous to k in equation (9). The raw data from the *FFT buffer* was also sent to a First in First out (FIFO) IP core[10] to store the raw data whilst the FFT block processed the dequeued samples. The processed data from the *FFT bin counter* and the data from the FIFO was then sent to another custom IP core, *FFT raw merger*. This IP took the two data streams and handled the valid- and ready signals as well as concatenating the size of the data streams into a 128-bit block, sent to an asynchronous FIFO for the DMA. This FIFO takes the PL- and PS clocks as inputs, and handles when to buffer and when to dequeue. Furthermore, it handles any delay in writing to the DDR3 memory of the PS, which is introduced by the DMA. It also prevents loss of data in case of a backlog in the stream; the DMA can only send data with a burst rate of 256 packets per burst.

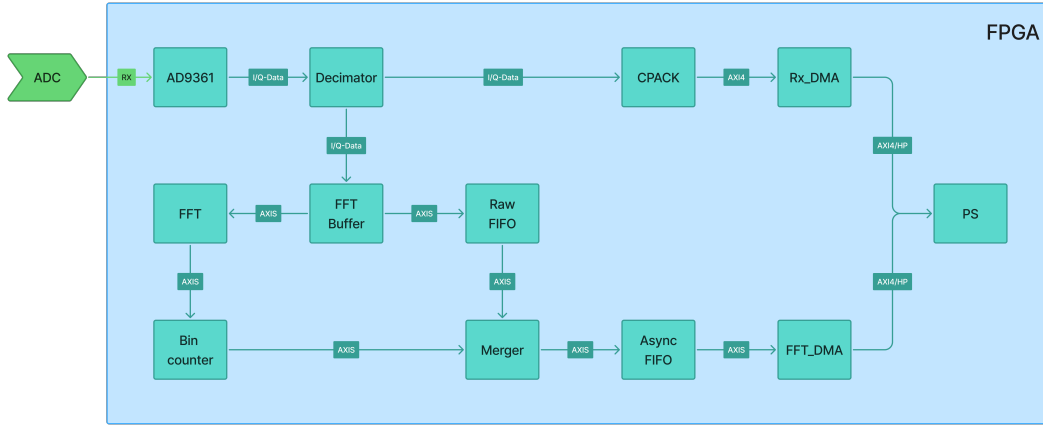


Figure 5: Signal path with FFT implemented on the FPGA

Custom IP blocks

As previously discussed, in order to transfer the I/Q stream to the FFT and thereafter, the processed samples to memory, three IP blocks were created.

FFT buffer

This IP block was created to make sure that a coherent window is saved from the I/Q stream and then sent to the FFT. It does this by having a buffer of registers, the length of the window size. The buffer has two states. The first state, "buffering state", listens for a valid signal as well as data, accumulating the window size into the buffer, and once reaching the desired window size N , transitions to its sending state. In the sending state, the valid flag is high, and the buffer transfers data to the two outputs as soon as the receiver is ready. This handshake is done to make sure the receiver is not busy. In this case, the receivers are the FFT and the FIFO buffer. Once the whole buffer has been sent, the *tlast* signal is set.

FFT bin counter

This IP block was created to make sure that the data being sent out from the FFT is labelled with the bin number it corresponds to. The block counts the valid signals from the FFT axis output, and checks to see if this equals N . If so, the counter state resets, and outputs a *tlast* signal. The FFT data is now split up, sent along with bin number, and a *tvalid* signal.

FFT raw merger

To write all the relevant data to memory, the data should be presented in one bit string. The *FFT raw merger* takes both inputs and combines them together to a 128 bit long data output along with padding to make sure that the data format complies with the axis standard. The block also takes flags as input from the FIFO buffering the raw IQ samples, and the DMA FIFO. These flags prevent the two blocks from receiving a ready signal should the blocks not be ready. This is done to make sure that the data written to the memory through the DMA will be valid data blocks from both the FFT and the raw FIFO.

Device drivers

To make the data available in software, the processed data needs to be written to the memory of the board, where it will be accessed using the LibIIO interface. To expose the *FFT DMA* to the device, the DMA has to be added to the Linux device tree. Along with this, a new Industrial input output (IIO) context and device has to be created to be able to read directly from the that DMA. The repository being used made it difficult to create a new IIO device to read the data directly from a custom IP core, in this case the DMA. A solution of redirecting the existing IIO device *cf-ad9361-lpc* was used to read the relevant data. This was done by redirecting the IIO device to read data from *FFT DMA* instead of the original *RX DMA* that is only writing the raw data. This change was done in the device tree and led to the output going from 32 bit I/Q data to the 128 bit string with all relevant data seen in Figure 6. The device tree entry for the *RX DMA* was used as reference when adding the *FFT DMA*. The hardware address and interrupt signal were changed.

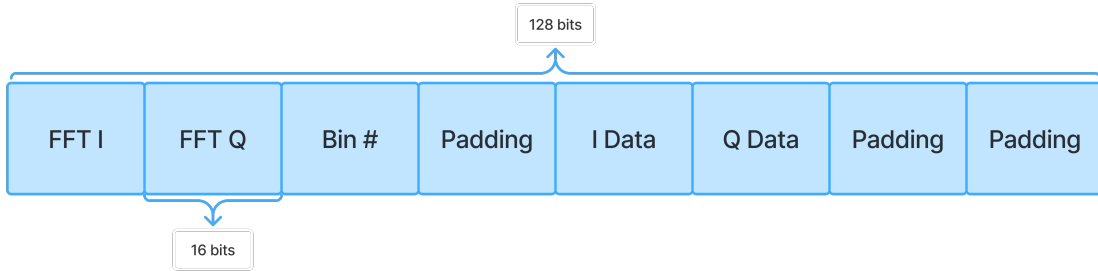


Figure 6: Data string with Processed data, bin number, raw I/Q data as well as padding

Software design

The following is a description of some notable libraries used, and what problem they solve in our project. In addition, core technology used in the software is also described.

The server- and UI code is written in Python. Libraries used and their applications are

- Numpy - math and sample data processing
- QT - GUI and plots
- Socket - stock library for sockets
- Json - stock library for the json data format

The client code is written in C. Libraries used and their applications are

- LibIIO - communication with iio devices, such as the SDR
- Socket - stock library for sockets

- Cjson - lightweight library allowing json processing in C

We explain some of these in further detail as they form a foundation of the project.

Socket communication

Communication with and from the server is made possible with socket interfaces.

A socket is essentially an interface for low-level network communication between a host (server) and recipient (client), providing access to standard transport layer communication protocols such as TCP and UDP within the OSI-model (ISO/IEC 7498)[11]. The TCP socket is used for creating a bidirectional communication pipeline between the server and each client that connects to the server. This means a socket connection allows both the client and server to send and receive data simultaneously. In TCP, connections are persistent, meaning after a connection handshake is established, the parties will recognize each other, and *network packets* may then proceed to be sent over the pipe until the interface is closed. Internally, a socket is no different from a file; on UNIX-like systems, they are identified using a file descriptor, and read from / written to using standard system calls such as `read()` and `write()`. However, opening a socket, thus allowing for remote connection, is different from that of creating a file. Figure 8 illustrates the different states of a socket.

In our case, we use a socket for inter-process communication over LAN for development, but this may be extended to over the internet in production.

LibIIO

Industrial input output (IIO) is a communication interface for sensors and devices that perform analog-to-digital conversion and digital-to-analog conversion. IIO is common, so drivers exist in the Linux kernel. To make it easier to work with these, the project uses LibIIO, and specifically the C Application programming interface (API) thereof, to ease working with software.

There are five stages to using IIO:

1. Context creation - method to establish connection, could be an IP address.
2. Devices - connect to a device, such as the receiver.
3. Channels - connect to a channel of a device, and enable the channel.
4. Interchange data - Read/write to attributes of a channel, or create a data stream.
5. Destroy - When done, free their memory to prevent a memory leak.

On the software side of the project, this schema is used extensively when handling server requests and responses.

Data format

Data sent over a wire, or otherwise transferred, must be encoded prior to being sent, and then decoded upon being received. It quickly becomes unbearable to handle structured data that belongs together in one send request, which is prevalent whenever requests are made. JavaScript Object Notation (JSON) is a lightweight data-interchange format that solves this problem. It may not be the fastest alternative, but for our purposes, it works fine. It is easy to interpret and can be encoded/decoded. An example of JSON data that is used:

```
{
  "type": "cfg",
  "rx": {
    "bw": bw_text,
    "lo": lo_text,
    "fs": fs_text,
    "hardwaregain": gain_text
  }
}
```

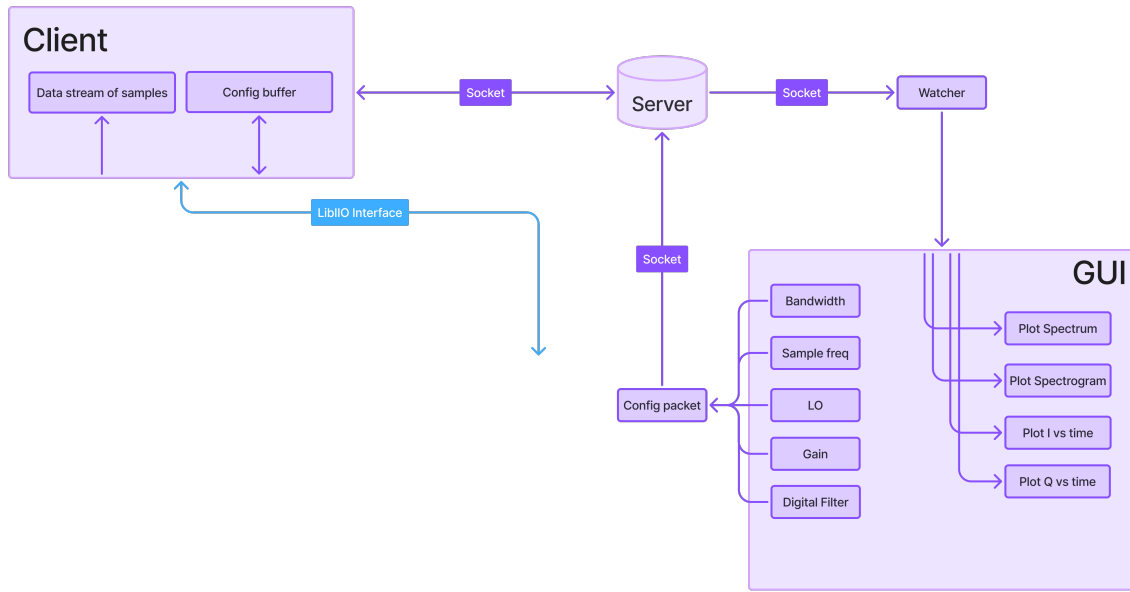


Figure 7: Flowchart of software design

```

}
// where the "*_text" variables store current inputs to text fields.

```

However, just sending the above user data in human-readable form as opposed to a byte string would garble it, and the recipient would then decode an unexpected format.

In addition to this, the socket appends control information to this data, making it a network packet. The data in this context is known as the payload. In figure 7, this packet is sent from an event handler in the GUI. Upon receiving this payload, the server directs it to the client. The client inspects its type and performs necessary writes using IIO. If the writes were successful, the client responds to the server with

```

{
    "type": "cfg_success"
}

```

The GUI has a thread called watcher, which polls the server to see if anything can be received. In this case, the above payload is received, and is used by the GUI to verify whether the new parameters were set. If they were not set, the GUI writes so to the execution log.

Threading and concurrency

In the current implementation, the server and GUI run in a single process. This process has three threads: the socket worker, the watcher, and the sweeper. The latter is used whenever a frequency sweep is performed in the spectrum analyser.

Whilst we have used constructs such as semaphores, the aforementioned threads do not wake up due to a semaphore. Instead, they poll endlessly, but do nothing if certain conditions are not met. This is not an issue currently, but semaphores or mutexes could be used to wake them up explicitly. We did not implement this because there are not any shared resources across the threads other than constructs which are already thread-safe, such as the packet queue [12].

The socket worker and watcher threads, however, use a semaphore to synchronise upon the first connection. Since the watcher is not the SDR client, it is distinguished as such using a semaphore. The semaphore blocks

the watcher until the socket worker has registered the connection. When registered, the semaphore is released, and the watcher continues.

We realised a possible future addition here. This functionality could be extended to work for multiple SDR-clients, however, some problems need to be addressed first. More on this in the last section.

API

Unfortunately, a capable SDR as the Model of SDR (ANTSDR e200), seems to not be very supported, and previous examples of work using the board is scarce or outdated. The firmware in the repository we successfully managed to install, works because it restores a previously removed patch from its submodules. Why it was removed in the first place is a mystery to us, albeit the C API for interfacing with the AD9361 (ad9361.h) is outdated and does not work as expected. Possibly due to the patch applying changes, upgrading the firmware.

Still, we needed to be able to communicate with the SDR via software, so time and effort went into writing abstractions for the low-level LibIIO constructs to perform necessary functionality. The way we see it, both bad and good came of this. The good is that it forced us to learn the barebones LibIIO interface, which is knowledge that can be applied to other transceivers with an ADC or Digital-to-analogue converter (DAC). The bad is that it felt like that time could be better spent on helping each other out, or on more interesting things.

The API we have written should be able to work for interfacing with other transceivers too, since we wrote it with the intention to not solely target the AD9361, as support for it seems to be minimal.

More work could be done on the API as it implements only the necessities, but it is available in the core project code.

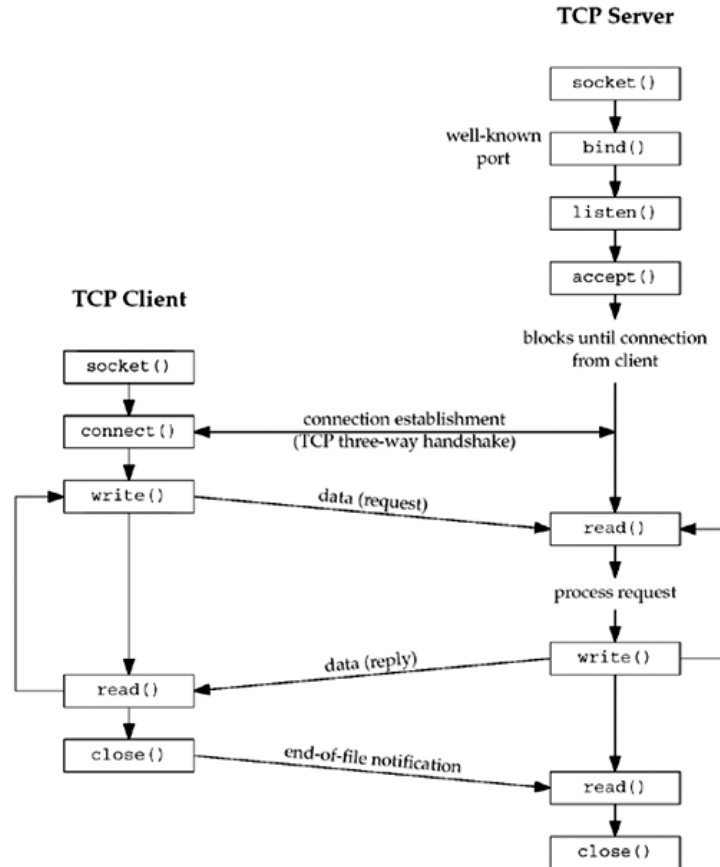


Figure 8: A flowchart depicting the flow of network packets over a TCP socket

Discussion

The project was a learning experience, and the tasks were varied, so you did not feel there was too much of the same thing, although you could get stuck at times.

A big hurdle of the project was finding firmware to build upon. Assembling firmware from scratch was deemed unfeasible given the tight time window of the project. Different repositories were explored as an option to have as a base for the project. These repositories were opened and would have provided a baseline to build the application on the SDR. The repositories were however outdated and support for them had stopped since a few years back. This led to the decision of using Microphase git repository. This was a working firmware but much harder to make changes to the firmware, but it was the only feasible firmware to have as a baseline for this project.

There also stems an issue from LibIIO. Writing kernel drivers using the IIO subsystem proved to be difficult. Especially writing drivers for existing firmware. The software ran into problems using LibIIO, such as bugs reading from memory. There was also some work to make sure the data being received was displayed correctly on the GUI as well as creating a sweep for analysing a larger spectrum than the SDR bandwidth.

The decision to make the Microphase git repository firmware the baseline of this project affected several more aspects of the project. It was difficult to make changes to the configurations file of the project to introduce new drivers. These new drivers were needed to make the iio context and driver to present the data of the FFT done on the hardware level. This led to a workaround where the data written by *FFT DMA* was read through the existing IIO device *cf-ad9361-lpc*, this solution works to read the needed data but is not optimal. This workaround should in future project be looked at and a more suitable solution by creating a new IIO context and IIO device for spectrum analyser should be made.

Result

The result is a PoC. The spectrum analyser is functional, filters and FFT are implemented at hardware level on the FPGA of the Zynq-7000 processor. The processed data could then be sent to the GUI over sockets while communicating with a server. The GUI successfully receives the reconstructed signal data and plots them. The host can also set parameters on the SDR from the GUI.

Figure 9 shows an overview of the entire spectrum analyser.

The System works by taking the digitised I/Q signals from the ADC on the hardware level. The FFT size used is $N = 2^{10}$, which means there are 2^{10} different frequency bins that each sample could correspond to. The Cooley–Tukey FFT algorithm[13] seems to be the version of FFT algorithm implemented in the Vivado libraries, and it works on sample lengths which are powers of two. For more resolution, increase the power of two. The ADC then sends this data along with the I/Q raw data to memory, which can be read by the client. The client is run on a host that is physically connected to the SDR via Ethernet. The client program gathers the data from the memory of the SDR and stores it in a temporary buffer before it is sent to the server, which in turn sends it to the GUI. The GUI is a separate program that connects to the server and in extension the client socket. From the GUI the data from the SDR is presented in different plots both for the raw time domain data as well as the processed FFT data showing the frequencies active in the current spectrum. The GUI is also used to set the parameters for the SDR controlling what LO frequency the SDR will be looking at and the bandwidth it will sample. The GUI is also using this function to perform a sweep of a larger spectrum by rapidly changing the LO frequency and saving the frequency response into a larger spectrum.

Images

Future Additions

Future addition to this project is to solve the issues caused by using the current repository for firmware and building the boot and OS of the SDR. A solution of writing a own no-OS using the firmware layout of the FPGA would result in that the client code could be easily implemented on the SDR and communicate on its

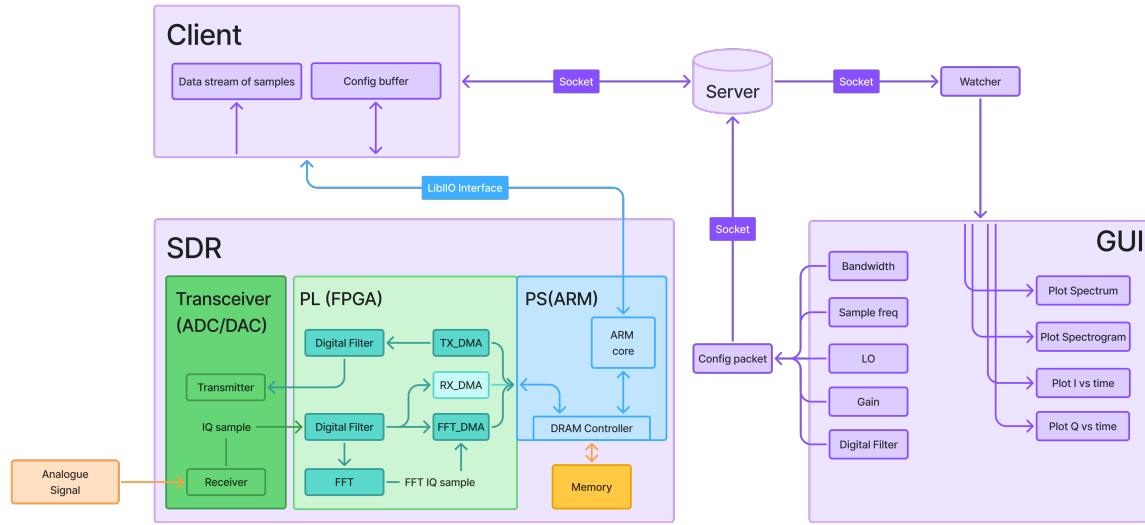


Figure 9: Overview of the final Spectrum analyser.

own with the server. This has the benefits of making the spectrum analyser much more mobile. This would also make it easier to create a new iio context that will read the data from the FFT cleaner.

Windowing

The loopback mode on the AD9361 is used for debugging purposes, specifically to test software-defined signals by loading such a signal on the transmitter and buffering that to the receiver.

As of now, enabling loopback mode and loading a sum of sines of different frequencies causes the spectrum to correctly plot its frequency components, although there are artifacts around these 12. Loading a chirp signal causes artifacts to appear in the spectrum, making it invalid 13.

To make sure the FFT only shows frequency content that is in the original signal, windowing could be applied. A Window function such as a Hamming Window could aid in decreasing aliasing and lowering the noise floor [14].

This windowing has to be applied in the time domain before conversion to the frequency domain. Thus, at the hardware level, the I/Q samples need be multiplied by the window function before they are given to the FFT. This should be possible to implement as a FIR filter.

Multi-client resolution

1. Communication is done via Ethernet - swap to wireless alternatives.
2. The server does distinguish between SDR clients, but the parameters set from the GUI are sent to every connected SDR.
3. Real time streaming - the receiver of every SDR will stream, causing the packet queue to fill up faster and the plots such as signal and spectrum will present samples alternating between each SDR.

A change that could be incorporated to resolve these issues would be to have an option in the GUI to select which SDR to make changes for; that is, each option populates the input boxes and plots, with data only from that SDR. When switching from one SDR to another, garbage collect the gathered resources associated with the previous SDR.

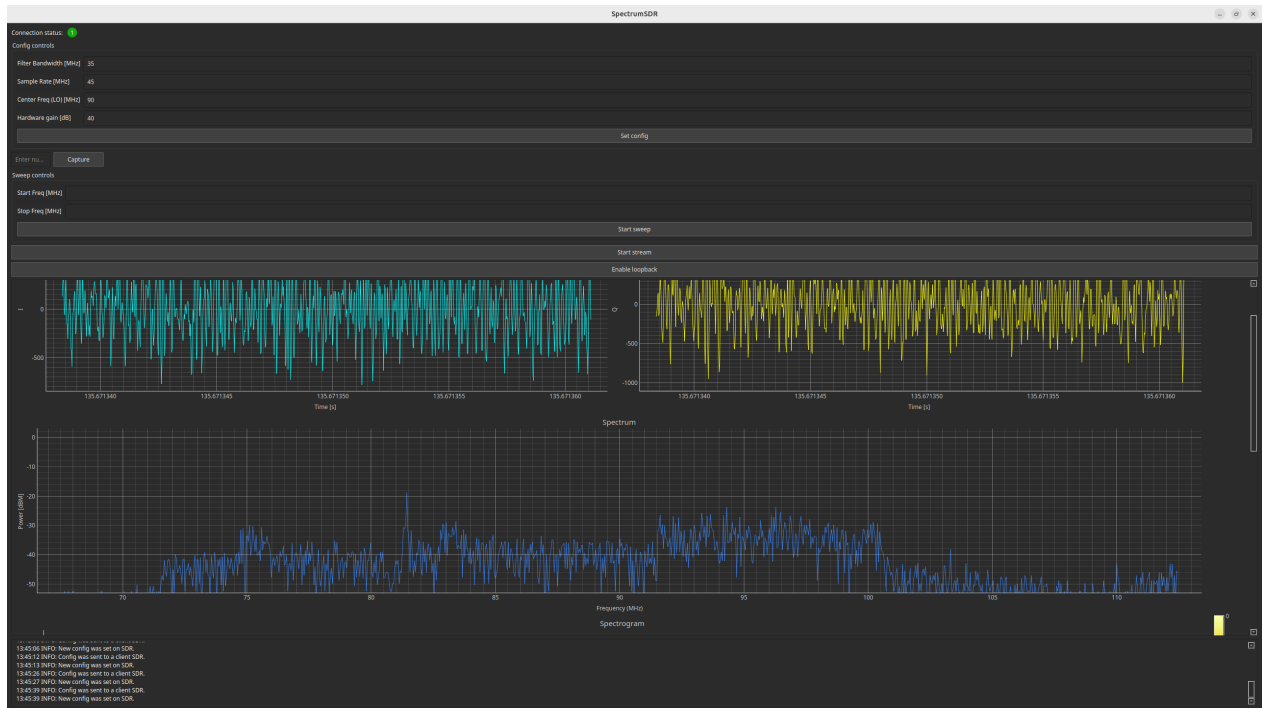


Figure 10: The I/Q and spectrum plots

```

13:45:13 INFO: New config was set on SDR.
13:45:26 INFO: Config was sent to a client SDR.
13:45:27 INFO: New config was set on SDR.
13:45:39 INFO: Config was sent to a client SDR.
13:45:39 INFO: New config was set on SDR.
13:53:28 INFO: Capturing 30 pkts in real time...
13:53:28 INFO: Captured 30 pkts → shot_2025-08-14_13:53:20.json

```

Figure 11: Log

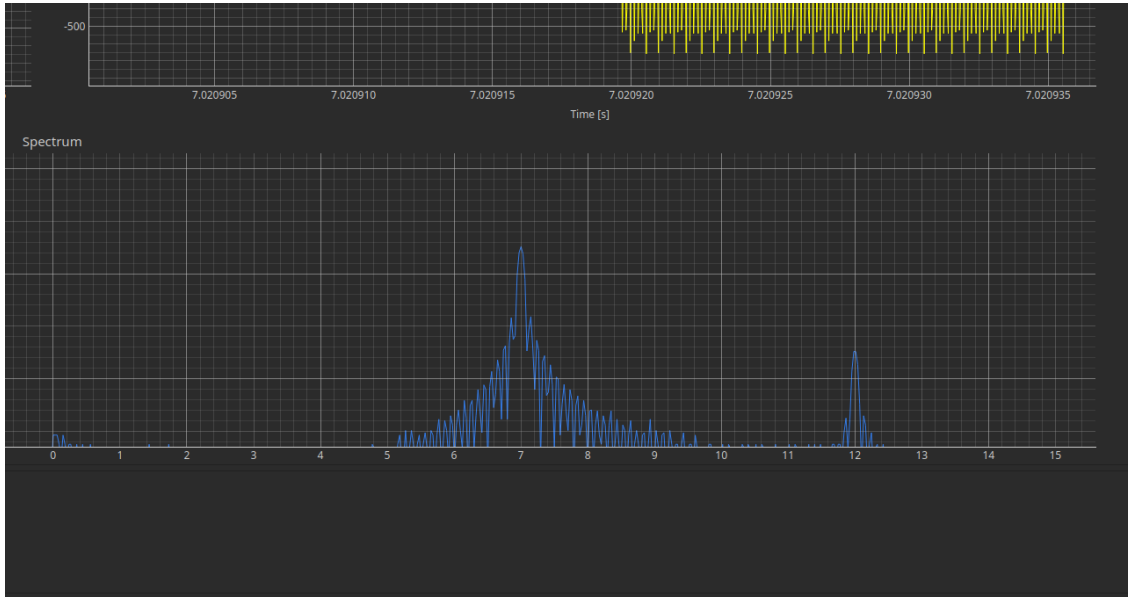


Figure 12: Sine wave with high amplitude at 7.0 MHz and lower amplitude at 12 MHz.

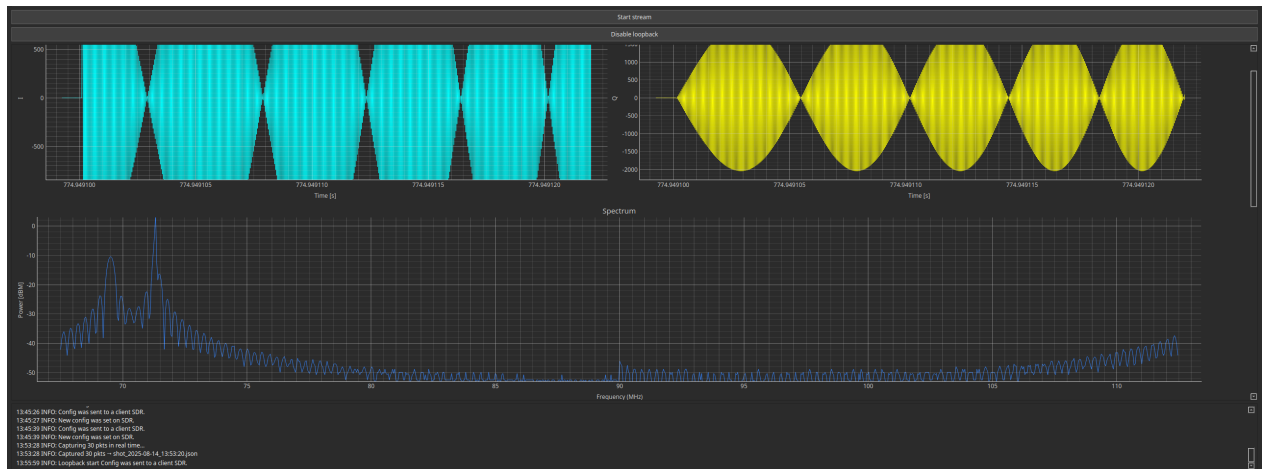


Figure 13: Chirp artifacts

Another similar functionality would be to let spectators connect to a specific client. Multiple people may need to view the plots simultaneously, but to prevent spectators from writing to a client, an ownership protocol could be implemented that only allows the host who started the client program, to write to the SDR, whilst spectators are allowed to read the data stream. If two or more hosts should be allowed to write very closely together, then this needs to be addressed through synchronisation constructs. This is because the SDR IIO connection is shared, so race conditions must be taken into account.

References

- [1] Wikipedia, *Nyquist-Shannon sampling theorem*, https://en.wikipedia.org/wiki/Nyquist-Shannon_sampling_theorem, Accessed: Aug, 2025.
- [2] Wikipedia, *Fast Fourier transform*, https://en.wikipedia.org/wiki/Fast_Fourier_transform, Accessed: Aug, 2025.
- [3] AMD, Inc., *AXI Basics 1 – Introduction to AXI*, https://adaptivesupport.amd.com/s/article/1053914?language=en_US, Accessed: Aug, 2025.
- [4] AMD, Inc., *AXI4-Stream Interfaces*, <https://docs.amd.com/r/en-US/ug1399-vitis-hls/AXI4-Stream-Interfaces>, Accessed: Aug, 2025.
- [5] MicroPhase Technologies, *antsdr-fw-patch: Firmware Patches for ANTSDR Devices*, <https://github.com/MicroPhase/antsdr-fw-patch>, Accessed: Jul, 2025.
- [6] Analog Devices, *util_cpack AXI Core Documentation*, https://wiki.analog.com/resources/fpga/docs/util_cpack, Accessed: Jul, 2025.
- [7] Analog Devices, *axi_dmac AXI Core Documentation*, https://wiki.analog.com/resources/fpga/docs/axi_dmac, Accessed: Jul, 2025.
- [8] AMD Xilinx, *FIR Compiler v7.2 LogiCORE IP Product Guide*, https://www.xilinx.com/support/documents/ip_documentation/fir_compiler/v7_2/pg149-fir-compiler.pdf, Accessed: Jul, 2025, 2020.
- [9] AMD Xilinx, *Fast Fourier Transform v9.1 LogiCORE IP Product Guide*, https://www.xilinx.com/support/documents/ip_documentation/xfft/v9_1/pg109-xfft.pdf, Accessed: Jul, 2025, 2020.
- [10] AMD, *FIFO Generator LogiCORE IP Product Guide (PG057)*, <https://docs.amd.com/v/u/en-US/pg057-fifo-generator>, Accessed: Jul, 2025.
- [11] ISO: Global standards for trusted goods and services, *ISO/IEC 7498-1:1994*, <https://www.iso.org/standard/20269.html>, Accessed: Jul, 2025.
- [12] Python Docs, *collections - Container datatypes*, <https://docs.python.org/3/library/collections.html>, Accessed: Aug, 2025.
- [13] Wikipedia, *Cooley-Tukey FFT algorithm*, https://en.wikipedia.org/wiki/Cooley-Tukey_FFT_algorithm, Accessed: Aug, 2025.
- [14] Wikipedia, *Window function*, https://en.wikipedia.org/wiki/Window_function, Accessed: Aug, 2025.